



Applied Deep Learning
O. Azencot
Spring 2026

Assignment 4: **Convolutional Neural Networks**

Deadline: June 25, 2026 at 17:00

In this assignment, you will implement a convolutional neural network (CNN) for image classification and explore the internal representations learned by the network using deconvolutional techniques.

The goals of this assignment are:

- training CNNs for image classification,
- understanding latent feature representations,
- reconstructing images from intermediate representations,
- interpreting the behavior of learned convolutional filters.

Submission guidelines:

- Work in pairs only.
- Submit the following two files via Moodle:
 - `report.pdf`
 - `code.zip`
- The report should contain all requested figures, explanations, and discussions.
- Do not include source code inside the report.

For questions, please use the course Slack channel before contacting the instructor by email.

1 Background

In Lecture 7, we studied convolutional neural networks (CNNs) and their differences from fully connected neural networks. During the coding session, we demonstrated a simple CNN for image classification on the CIFAR-10 dataset.

The CIFAR-10 dataset consists of RGB images of size 32×32 belonging to ten object categories.

In this assignment, you will reproduce the basic classification pipeline shown in class, extend it with a reconstruction branch, and investigate the representations learned by the network.

Latent features. A latent feature is an intermediate representation produced by the network. Given an image

$$x \in \mathbb{R}^{3 \times 32 \times 32},$$

we denote by

$$z^{(l)}$$

the output of the l -th convolutional block.

For example, in the CNN discussed in class, each block consists of:

- convolution,
- ReLU activation,
- pooling.

Applying the first and second convolutional blocks produces latent representations

$$z^{(1)}, \quad z^{(2)}.$$

These latent representations often capture meaningful information such as edges, textures, colors, and object parts.

Deconvolutional models. One way to visualize latent representations is through deconvolutional networks.

The basic idea is to approximately invert the operations performed by the encoder:

$$\begin{aligned} \text{pooling} &\rightarrow \text{unpooling} \\ \text{convolution} &\rightarrow \text{transposed convolution} \\ \text{ReLU} &\rightarrow \text{ReLU} \end{aligned}$$

The modified network should jointly perform two tasks:

1. image classification,
2. image reconstruction.

Given an input image x , the network produces

- class predictions $\tilde{y}$,
- reconstructed image $\tilde{x}$.

The training objective is

$$\mathcal{L} = \mathcal{L}_{\text{ce}}(\tilde{y}, y) + \lambda \mathcal{L}_{\text{rec}}(\tilde{x}, x),$$

where

$$\mathcal{L}_{\text{rec}} = \frac{1}{3} \sum_{i=1}^3 \|\tilde{x}_i - x_i\|_F^2$$

and $\lambda > 0$ controls the tradeoff between classification and reconstruction quality.

2 Tasks

1. CIFAR-10 Classification

Follow the PyTorch CIFAR-10 tutorial:

https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

Implement the CNN and training procedure described in the tutorial.

Your report must include:

- training accuracy,
- test accuracy,
- several test images together with their predicted labels,
- the per-class accuracy table from the tutorial.

You are not required to reproduce exactly the same numerical results as the tutorial.

2. Deconvolutional Model

Extend the CNN from Task 1 by adding a decoder network that approximately reconstructs the input image.

The decoder should contain:

- unpooling layers (or a reasonable alternative),
- transposed convolutions,
- nonlinear activations.

The decoder should produce an image

$$\tilde{x} \in \mathbb{R}^{3 \times 32 \times 32}.$$

Train the model using the combined loss

$$\mathcal{L} = \mathcal{L}_{\text{ce}} + \lambda \mathcal{L}_{\text{rec}}.$$

You are free to choose λ .

Briefly explain your choice in the report.

Your report must include:

- train and test accuracy curves,
- final test accuracy,
- value of λ ,
- 2–3 original images,
- their corresponding reconstructions.

3. Latent Representation Analysis

Select:

- one image from the training set,
- one image from the test set.

For the first convolutional layer:

- Compute the latent representation $z^{(1)}$.
- For each channel separately:
 - keep only that channel,
 - set all remaining channels to zero,
 - reconstruct the image using the decoder.

Repeat the same procedure for any three channels of the second convolutional layer.

Your report must include:

- the original images,
- the reconstructed images obtained from the selected channels,
- a discussion of the visual patterns captured by the channels.

In particular, discuss whether different channels appear to focus on:

- edges,
- colors,
- textures,
- object parts,
- or other meaningful structures.

3 Additional Comments

1. Do not include source code inside the report.
2. A significant portion of the grade is based on clarity, organization, readability, and quality of analysis.
3. Keep the report concise. Aim for at most 8 pages.
4. All figures should include captions.
5. When presenting reconstructions, place the original image next to the reconstruction whenever possible.
6. If computational resources are limited, you may reduce the dataset size by a factor of 10 (6,000 training images instead of 60,000). If you choose this option, clearly state it in your report.
7. Training to perfect convergence is not required. Correct methodology and thoughtful analysis are more important than obtaining the highest possible accuracy.
8. To implement the decoder, see:
 - <https://pytorch.org/docs/stable/generated/torch.nn.ConvTranspose2d.html>
 - <https://pytorch.org/docs/stable/generated/torch.nn.MaxUnpool2d.html>
9. An example implementation of a deconvolutional network can be found at:

<https://github.com/huybery/VisualizingCNN/tree/master>